

# TABLE DES MATIÈRES

|                                                        |    |
|--------------------------------------------------------|----|
| Introduction.....                                      | 3  |
| 1. Description de l'application.....                   | 4  |
| 1.1 Présentation Générale.....                         | 4  |
| 1.2 Fonctionnalités .....                              | 4  |
| 1.3 Architecture Technique .....                       | 4  |
| 1.3.1 Backend .....                                    | 4  |
| 1.3.2 Frontend .....                                   | 5  |
| 1.3.3 Conteneurisation .....                           | 6  |
| 2. Hébergement du Code Source sur<br>GitHub .....      | 8  |
| 3. Jenkins - Automatisation du pipeline<br>CI/CD ..... | 9  |
| Configuration du pipeline .....                        | 9  |
| Commandes utilisées dans le pipeline ...               | 10 |
| Avantages de l'intégration Jenkins .....               | 10 |
| 4. Déploiement avec Kubernetes .....                   | 12 |
| Création des ressources Kubernetes .....               | 12 |
| Gestion des mises à jour avec Kubernetes<br>.....      | 12 |
| Conclusion.....                                        | 15 |

# INTRODUCTION

---

Le Dans le cadre de la modernisation des outils internes de l'entreprise AI-Vision, nous avons conçu et déployé To-Do Vision, un gestionnaire de tâches dédié aux employés. Ce projet vise à offrir une solution sécurisée et adaptée aux besoins de l'entreprise, évitant ainsi l'utilisation d'outils tiers pouvant exposer des données sensibles.

L'objectif principal de ce projet était de :

- Développer une application web permettant aux employés de créer, gérer et suivre leurs tâches.
- Automatiser le déploiement avec un pipeline CI/CD via Jenkins, assurant une intégration et un déploiement continus.
- Déployer l'application sur un cluster Kubernetes pour garantir une scalabilité et une gestion efficace des conteneurs.

Pour mener à bien ce projet, nous avons utilisé les technologies suivantes :

- **Flask** pour le développement du backend.
- **SQLite** comme base de données pour stocker les tâches.
- **Docker** pour conteneuriser l'application et assurer sa portabilité.
- **Jenkins** pour automatiser l'intégration et le déploiement continus.
- **Kubernetes** (Minikube) pour la gestion du déploiement et l'orchestration des conteneurs.
- **GitHub** pour l'hébergement et le suivi du code source.

Ce rapport détaille les différentes étapes de la conception, du développement et du déploiement de To-DoVision, ainsi que l'automatisation du pipeline CI/CD et la mise en production sur Kubernetes.

# 1. DESCRIPTION DE L'APPLICATION

---

## I.1 Présentation Générale

L'application To-DoVision est un gestionnaire de tâches développé pour l'entreprise AI-Vision. Elle vise à offrir aux employés un outil sécurisé et dédié pour organiser leurs tâches quotidiennes, sans dépendre de services tiers qui pourraient compromettre la confidentialité des données.

## I.2 Fonctionnalités

L'application offre plusieurs fonctionnalités essentielles pour la gestion des tâches :

- **Ajout de tâches** : L'utilisateur peut ajouter une tâche en saisissant un texte dans le formulaire prévu à cet effet.
- **Validation/Invalidation de tâches** : Une tâche peut être marquée comme complétée ou remise à l'état initial.
- **Suppression de tâches** : Une tâche peut être définitivement supprimée.
- **Affichage des statistiques** : Un compteur indique le nombre de tâches effectuées et le nombre total de tâches.

L'interface utilisateur est conçue avec HTML, Bootstrap, et du CSS personnalisé, offrant ainsi une expérience simple et fluide.

## I.3 Architecture Technique

L'application repose sur une architecture Flask + SQLite et est conteneurisée avec Docker et Docker Compose.

### I.3.1 Backend

- Framework : Flask (Python 3.9)
- Base de données : SQLite
- Routes principales :

/ : Affichage des tâches

/add : Ajout d'une tâche (POST)

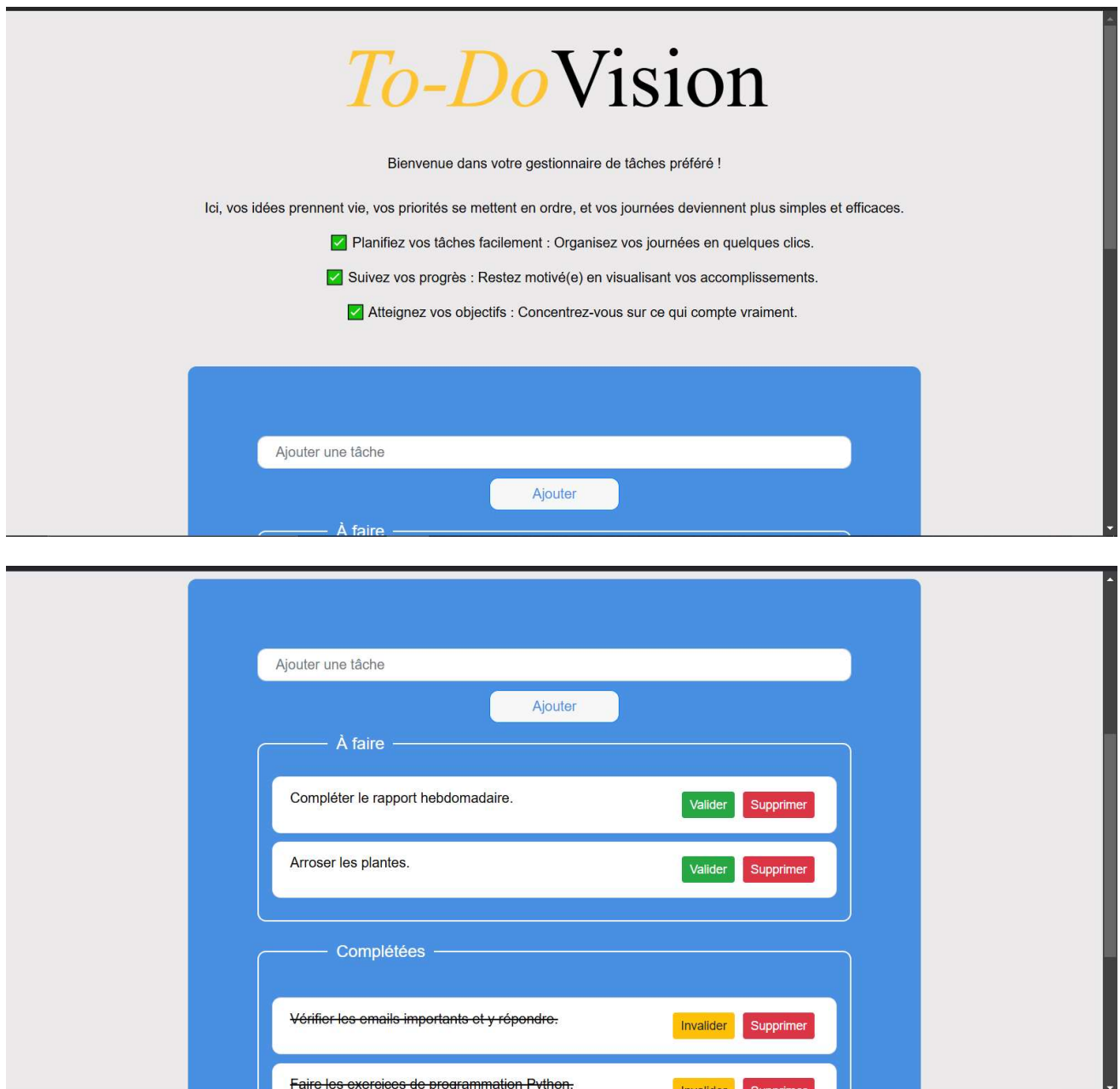
/delete/<int:task\_id> : Suppression d'une tâche

/complete/<int:task\_id> : Marquage d'une tâche comme complétée

/incomplete/<int:task\_id> : Remise d'une tâche à l'état incomplète

## I.3.2 Frontend

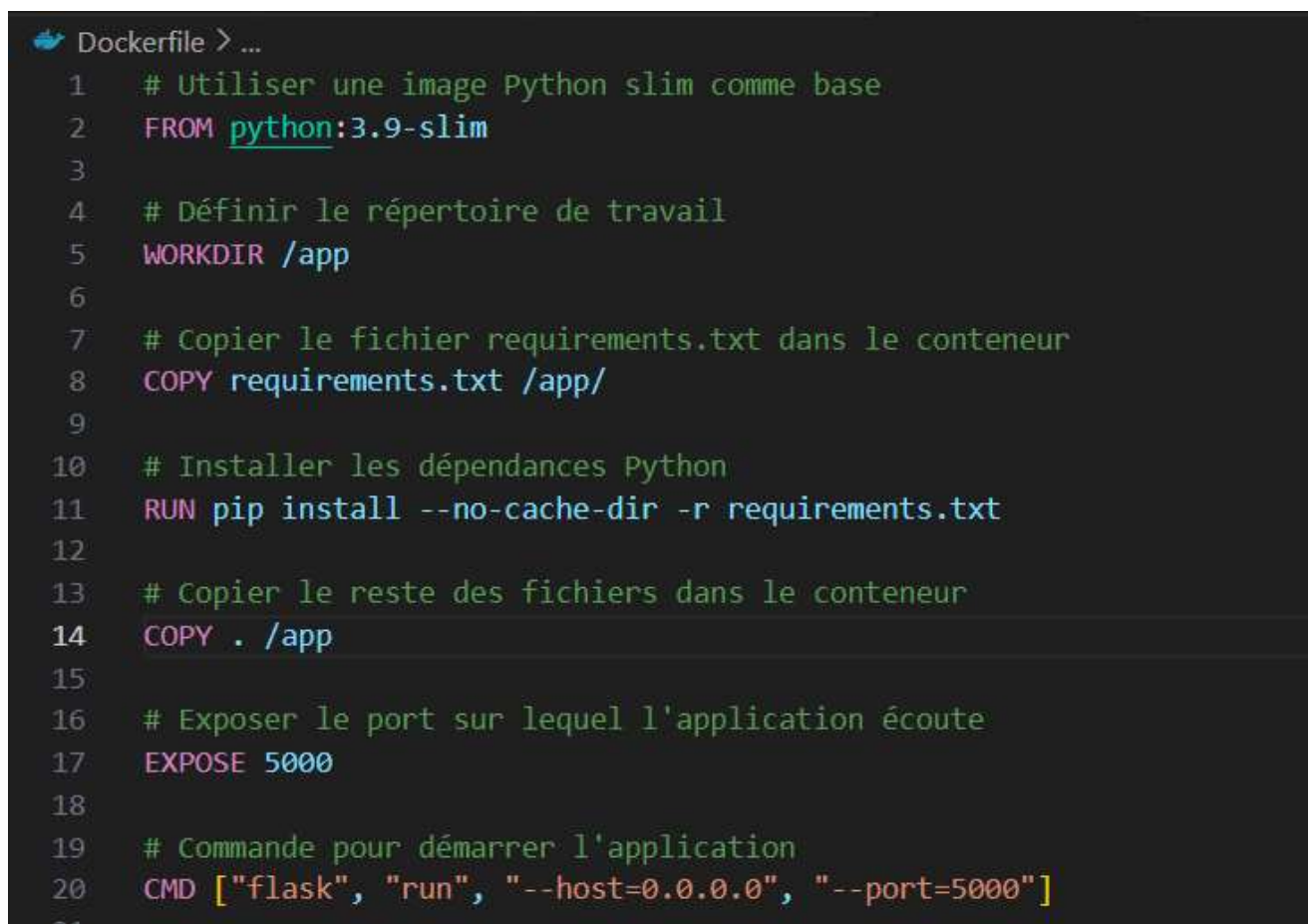
L'application est dotée d'une interface développée en HTML, CSS et Bootstrap, garantissant une navigation fluide et intuitive.



### I.3.3 Conteneurisation

L'application est déployée sous forme de conteneur Docker avec la structure suivante :

- Dockerfile :
  - Utilisation d'une image Python slim
  - Installation des dépendances via requirements.txt
  - Exposition du port 5000
  - Lancement du serveur Flask

A screenshot of a code editor showing a Dockerfile. The editor has a dark background with light-colored text. The Dockerfile content is as follows:

```
Dockerfile > ...
1  # Utiliser une image Python slim comme base
2  FROM python:3.9-slim
3
4  # Définir le répertoire de travail
5  WORKDIR /app
6
7  # Copier le fichier requirements.txt dans le conteneur
8  COPY requirements.txt /app/
9
10 # Installer les dépendances Python
11 RUN pip install --no-cache-dir -r requirements.txt
12
13 # Copier le reste des fichiers dans le conteneur
14 COPY . /app
15
16 # Exposer le port sur lequel l'application écoute
17 EXPOSE 5000
18
19 # Commande pour démarrer l'application
20 CMD ["flask", "run", "--host=0.0.0.0", "--port=5000"]
21
```

- Docker Compose :
  - Déploie le service Flask
  - Monte un volume pour la persistance des fichiers
  - Définit les variables d'environnement (FLASK\_ENV)

 docker-compose.yml

▶ Run All Services

1 services:

▶ Run Service

2 web:

3 build: .

4 ports:

5 | - "5000:5000"

6 environment:

7 | - FLASK\_ENV=development

8 | - DATABASE\_URL=sqlite:///tasks.db

9 volumes:

10 | - ./app

11 command: flask run --host=0.0.0.0 --port=5000

12

## 2. HEBERGEMENT DU CODE SOURCE SUR GITHUB

---

Pour assurer une collaboration efficace et un suivi rigoureux des modifications, nous avons hébergé le code source de To-DoVision sur GitHub. Ce choix a permis de centraliser le développement, de suivre l'historique des versions et de garantir une meilleure gestion des contributions au sein de l'équipe.

Nous avons adopté une organisation simple avec deux branches principales :

- master : Cette branche servait de zone de travail principale où les nouvelles fonctionnalités et corrections de bugs étaient développées et testées. Toutes les contributions y étaient poussées avant validation.
- main : Réservée au code stable et validé, cette branche contenait uniquement les versions fonctionnelles et prêtes à être déployées.

Le workflow suivi était le suivant :

- Développement et tests sur la branche master.
- Une fois une fonctionnalité validée, elle était fusionnée dans la branche main.
- La branche main servait ensuite de référence pour le déploiement de l'application.

Grâce à cette structuration, nous avons pu garantir la stabilité du projet tout en permettant une flexibilité dans le développement des nouvelles fonctionnalités.

[Lien Github du dépôt](#)

## 3. JENKINS - AUTOMATISATION DU PIPELINE CI/CD

---

Dans le cadre de notre projet To-DoVision, nous avons mis en place un pipeline d'intégration et de déploiement continu (CI/CD) avec Jenkins. L'objectif principal était d'automatiser le processus de construction, de test et de déploiement de l'application afin d'assurer une livraison rapide et fiable des mises à jour à l'environnement de production.

### Configuration du pipeline

Le pipeline Jenkins a été configuré à l'aide d'un fichier Jenkinsfile contenant plusieurs étapes clés :

**Clonage du code source** : La première étape consiste à récupérer automatiquement le code du dépôt GitHub à chaque changement. Cela permet à Jenkins de toujours travailler avec la dernière version du code source.

**Construction de l'image Docker** : Ensuite, Jenkins utilise Docker pour construire l'image de l'application à partir du Dockerfile présent dans le répertoire du projet. L'image est étiquetée avec un nom spécifique, ce qui permet de suivre les versions de l'application déployée.

**Déploiement avec Docker Compose** : Après la construction de l'image, le pipeline lance le déploiement de l'application en utilisant Docker Compose. Cela permet de gérer plusieurs conteneurs Docker (par exemple, l'application elle-même et la base de données) dans un environnement local ou de test.

**Nettoyage après déploiement** : Une fois le déploiement effectué, une étape de nettoyage est incluse pour arrêter et supprimer les conteneurs et réseaux Docker afin de maintenir un environnement propre pour les exécutions futures du pipeline.



Jenkins

Tableau de bord > to-do\_list\_web\_app > #21 > Pipeline Console

✓ < Build #21

Succès 15 mn ago in 31 s

- ✓ Checkout SCM
- ✓ Cloner le dépôt GitHub
- ✓ Construire les conteneurs avec Docker
- ✓ Déployer l'application avec Docker Compose
- ✓ Post Actions

Stage 'Post Actions'

- Started 15 mn ago
- Queued 0 ms
- Took 12 s
- Success
- Running on Jenkins
- [View as plain text](#)

**docker-compose -f %DOCKER\_COMPOSE\_FILE% down** 12 s

Windows Batch Script

```
C:\ProgramData\Jenkins\.jenkins\workspace\to-do_list_web_app>docker-compose -f docker-compose.yml down
```

Jenkins 2.494

## Commandes utilisées dans le pipeline

Le pipeline exécute les commandes suivantes pour effectuer les tâches ci-dessus :

```
bat 'docker build -t %DOCKER_IMAGE_NAME% .'
```

```
bat 'docker-compose -f %DOCKER_COMPOSE_FILE% up -d'
```

```
bat 'docker-compose -f %DOCKER_COMPOSE_FILE% down'
```

Ces commandes permettent de :

- Construire l'image Docker avec le nom spécifié.
- Lancer les conteneurs définis dans le fichier docker-compose.yml.
- Arrêter et supprimer les conteneurs après le déploiement.

## Avantages de l'intégration Jenkins

L'utilisation de Jenkins a permis d'automatiser plusieurs processus essentiels, ce qui a apporté plusieurs avantages :

**Cohérence et fiabilité** : Le pipeline garantit que les mêmes étapes sont suivies à chaque modification du code, réduisant ainsi les erreurs humaines et assurant la constance du déploiement.

**Rapidité** : L'automatisation permet des mises à jour fréquentes, tout en accélérant le processus de déploiement. De plus, la détection rapide des erreurs (par exemple, lors de la construction ou du déploiement) permet une correction rapide.

**Facilité de gestion** : L'intégration de Docker et Docker Compose dans Jenkins simplifie le déploiement et la gestion de l'application, même dans des environnements complexes avec plusieurs conteneurs.

## 4. DEPLOIEMENT AVEC KUBERNETES

---

Dans le cadre de la mise en production de l'application To-DoVision, nous avons utilisé Kubernetes pour orchestrer le déploiement et gérer la scalabilité. Kubernetes permet une gestion efficace des conteneurs en production, avec des fonctionnalités telles que l'auto-scaling, la mise à jour continue et la gestion de la disponibilité des applications.

### Création des ressources Kubernetes

Pour déployer l'application avec Kubernetes, nous avons créé un fichier de configuration `deployment.yaml` qui définit les ressources nécessaires, telles que les pods, les réplicas, et les services. Il contient les configurations pour déployer l'application dans un cluster Kubernetes.

#### Déploiement de l'application avec trois réplicas

Dans le fichier `deployment.yaml`, nous avons défini **trois réplicas** pour assurer la haute disponibilité de l'application. En cas de défaillance d'un pod, Kubernetes peut recréer automatiquement un autre pod pour garantir que l'application reste disponible. Ce nombre de réplicas peut être ajusté en fonction de la charge et des besoins en scalabilité.

#### Image avec Dockerhub

Dans le cadre du déploiement de l'application To-DoVision, une image Docker a été construite à partir du `Dockerfile` du projet, puis exportée et poussée sur Docker Hub. L'image, nommée `roy61/to-do_list_web_app-web:latest`, a été utilisée lors de la mise à jour du déploiement sur Kubernetes. Jenkins a automatisé l'exportation de l'image et son upload sur Docker Hub. Ensuite, l'image a été récupérée par Kubernetes pour être déployée sur le cluster.

#### Configuration du service pour exposer l'application

Nous avons configuré un Service Kubernetes pour exposer l'application à l'extérieur du cluster. Le service est de type `LoadBalancer`, ce qui permet de distribuer le trafic entrant entre les différents pods répliqués. Le service écoute sur le port 80, tandis que le conteneur de l'application écoute sur le port 5000.

### Gestion des mises à jour avec Kubernetes

Une fois que l'application est déployée sur Kubernetes, il est essentiel de pouvoir mettre à jour l'application sans temps d'arrêt. Kubernetes gère les mises à jour de manière fluide grâce à ses mécanismes de déploiement progressif.

## Modification du code source

Lorsque le code source est modifié (par exemple, une nouvelle fonctionnalité est ajoutée ou un bug est corrigé), l'image Docker de l'application est reconstruite et poussée vers le registre Docker.

## Redeploiement via `kubectl apply -f deployment.yaml`

Pour déployer la nouvelle version de l'application sur Kubernetes, nous utilisons la commande `kubectl apply`. Cette commande applique les modifications spécifiées dans le fichier `deployment.yaml`, y compris la mise à jour de l'image Docker dans les pods.

Kubernetes gère ensuite la mise à jour des pods en douceur, garantissant que certaines répliques de l'application restent actives pendant le déploiement, pour éviter tout temps d'arrêt.

## Vérification de la mise à jour

Après le déploiement, nous vérifions que les mises à jour ont été appliquées correctement et que l'application fonctionne comme prévu. Pour cela, nous utilisons les commandes suivantes :

*kubectl get pods* : Cette commande affiche la liste des pods en cours d'exécution. Nous pouvons y voir les nouvelles versions des pods.

## Vérification des services :

*kubectl get svc* : Cette commande permet de vérifier que le service expose correctement l'application.

En cas de problème, Kubernetes permet également de revenir à une version précédente de l'application en utilisant la commande suivante :

*kubectl rollout undo deployment/to-dovision-deployment* : Cela permet de restaurer l'état précédent du déploiement et d'assurer la stabilité de l'application.

royce@DESKTOP-G2Q30PB: ~/to-do\_list\_web\_app

GNU nano 7.2

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
        - name: flask-app
          image: roy61/to-do_list_web_app-web:latest
          ports:
            - containerPort: 5000
          env:
            - name: FLASK_ENV
              value: "development"
            - name: DATABASE_URL
              value: "sqlite:///tasks.db"
```

# CONCLUSION

---

Le projet To-DoVision a été un succès en termes de déploiement et d'automatisation grâce à l'intégration de plusieurs technologies de pointe, dont Docker, Kubernetes, et Jenkins. L'application a été déployée avec succès sur un cluster Kubernetes, offrant une gestion robuste des réplicas et des mises à jour continues. Le pipeline CI/CD a été entièrement automatisé avec Jenkins, assurant une livraison rapide et fiable des nouvelles versions de l'application. En utilisant Docker Hub pour stocker l'image de l'application, nous avons garanti une version cohérente et réutilisable de l'application à travers différents environnements.

Nous avons pu garantir la disponibilité et la scalabilité de l'application avec Kubernetes, et le processus de mise à jour a été simplifié grâce aux mécanismes de déploiement progressif offerts par Kubernetes. Le système de gestion des tâches, To-DoVision, est désormais prêt pour être utilisé dans un environnement de production et pourra être facilement adapté à de nouveaux besoins ou évolutions.